

RELAZIONE PROGETTO CCAI 2026

AGENTIC AI BLOG SUPPORT

Casaccio Agrippino 1000085471

Introduzione

L'obiettivo del progetto è quello di creare un sistema agentico di blog-support, in particolare si occupa della stesura degli articoli di un blog sulla Roma antica gestita da un museo a tema, in particolar modo sulla storia imperiale, ma non solo.

Il sistema inizialmente provvede a stilare la programmazione settimanale dei vari post, cadenzati in modo da avere tre post ogni sette giorni. La programmazione sarà poi sottoposta a revisione e, qualora dovesse essere accettata, si passerà alla stesura degli articoli.

Buona parte degli articoli saranno strutturati in modo discorsivo, presentando un piccolo testo in cui vengono date informazioni come una breve biografia di un imperatore, la descrizione di una battaglia, gli usi e costumi e le meraviglie architettoniche.

Gli articoli, così come la schedulazione, saranno redatti mediante chiamata ad un LLM e sottoposti anch'essi a revisione prima di essere pubblicati.

L'agente dispone di diversi tool a sua disposizione con i quali riesce ad utilizzare il Knowledge Graph e la ricerca web per la stesura dei post e la loro revisione, ma anche suggerire topic per la programmazione e sfruttare un modello Visual Transformer specifico per la stesura di particolari post.

Funzionamento

All'avvio del programma di blog-support, esso andrà inizialmente a verificare se è già memorizzato un EditorialPlanning: l'elenco dei post settimanali da pubblicare, composto dalla giornata, il topic ed una breve descrizione. Se è presente ed è possibile pubblicare, si passerà alla fase di pianificazione dei post, altrimenti il flusso totale terminerà.

In fase di pianificazione del post, verrà verificato ed estratto il topic corrispondente, dal quale verrà scritto un breve articolo, di circa una decina di righe. Un Large Language Model dedicato (e configurato in modo da avere output strutturato) riceverà via prompt sia il tema, sia la richiesta di andare a prelevare informazioni utili alla stesura sia tramite chiamata web su siti verificati, sia soprattutto sfruttando un Knowledge-RAG: Da un grafo KG infatti andrà a prelevare documenti (o parti di essi) in cui si parla di quell'argomento, la quale fingendo da ground truth serviranno da fonti da usare esplicitamente per il post.

Una volta prodotto il testo, il sistema estrae da esso i claim principali in formato JSON, che verranno controllati rispetto a quelli di post passati. Se risultano duplicati (similarità del 95%), verrà effettuata una ritrattazione automatica, rispedendo il flusso e l'assistente LLM lo riscriverà.

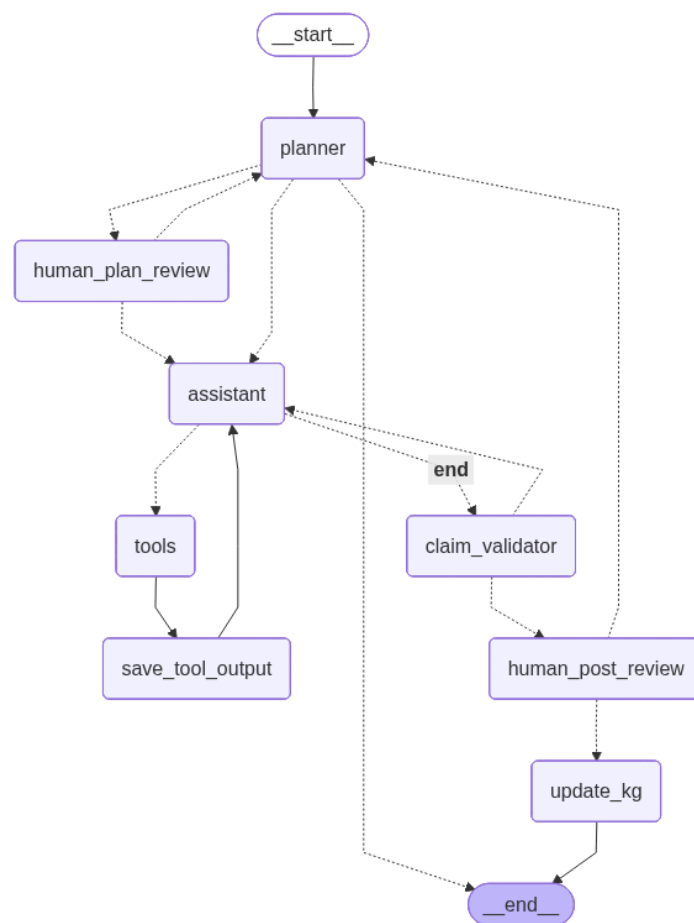
Il risultato valido sarà passato a revisione umana tramite interrupt con tre esiti possibili: “approvato” e pronto a essere pubblicato (e salvato nel KG); “modificato” con la riscrittura da parte dell’utente stesso o “rigettato”, rimandato indietro all’LLM con un piccolo feedback testuale il quale servirà al modello per riscrivere il post e continuare il ciclo.

Sono messi a disposizione dei tool per l’interazione col Knowledge Graph, sfruttare il K-RAG e interagire con un Visual Transformer fine-tunato di classificazione numismatica: sarà prelevata un’immagine di una moneta e verrà restituito l’imperatore in essa raffigurato e utilizzato in particolari post.

Qualora però, nella fase iniziale, non fosse presente un EditorialPlanning per quella settimana, prima di stilare i post all’LLM verrà posto il compito di creare una nuova schedulazione, seguendo il modello tracciato in precedenza. Anch’esso sarà sottoposto a revisione; se è “approve” verrà direttamente salvato nel KG e si passerà al post; altrimenti a seguito dei feedback ricevuti, verrà rimandato all’LLM e riscritto.

Tutta la logica di business è orchestrata tramite LangGraph utilizzando uno stato condiviso (AgentState) per tener traccia della cronologia dei messaggi e della reasoning trace.

Graficamente, per tener traccia del flusso, viene generato un diagramma di flusso, compilando il grafo e rappresentato tramite sintassi **Mermaid**.



In seguito una trattazione più approfondita dei vari passaggi.

Planning Requirements

All'avvio del programma, come detto in precedenza, esso andrà inizialmente a verificare se è già memorizzato (*tramite chiave*) un **EditorialPlanning**: si tratta dell'elenco dei post settimanali da pubblicare, strutturato in modo da essere composto dalla giornata di pubblicazione, il topic del post del giorno ed una breve descrizione dello stesso. Come giorni utili a scrivere i post sono stati scelti Lunedì, Mercoledì e Sabato.

Queste specifiche tecniche verranno inviate ad un Large Language Model che si occuperà di prendere questo prompt e provvedere alla creazione del Plan. LLM scelto è **MistralAI** (*mistral-small-latest*) data la sua natura free (dopo che in precedenza si è usata l'API di **Grok**, ritenuta però troppo limitata e soggetta a errori) e sarà invocato anche in fase di creazione del post. Il piano elaborato sarà poi soggetto a revisione da parte dell'utente, il quale può approvarlo o rifiutarlo, scrivendo un feedback di miglioramento e rimandando al LLM per essere rigenerato secondo i consigli acquisiti. Questo approccio è detto Human-in-the-Loop e sarà utilizzato parimenti anche per la creazione del post.

Per strutturare al meglio il plan, è stata utilizzata una classe **Pydantic** (usata per forzare il modello a fornire un output strutturato in JSON). Questo garantisce che i campi topic, justification e content siano sempre estratti correttamente dal grafo per poi essere usati successivamente. Se il piano esiste già, una funzione *is_publish_day()* verificherà se oggi è giorno di pubblicazione, reindirizzando ad un nodo di routing per gestire eventuale creazione di post o terminazione del programma qualora fosse giorno di riposo.

Knowledge Graph & K-RAG

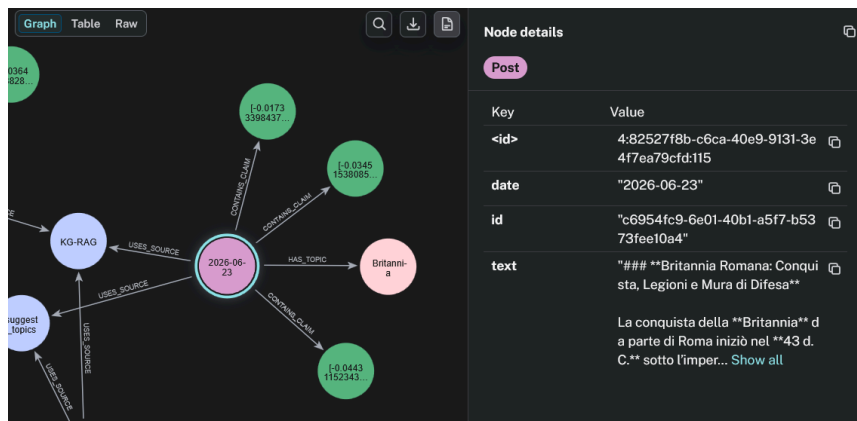
Tra le funzionalità più importanti che il sistema implementa vi è l'utilizzo di un **Knowledge Graph** editoriale sfruttando **Neo4j Aura**. Il grafo funge sia da archivio per post e documenti, ma viene attivamente utilizzato per generare il contenuto del post (da successivamente archiviare), implementando così anche la strategia del K-RAG.

Il KG e suo utilizzo

Il grafo viene direttamente invocato dal file principale tramite chiamata alla funzione *update_post_knowledge_graph()* del file relativo al KG stesso. Ogni qual volta che un post viene approvato dall'utente, questo grafo viene aggiornato creando o collegando nodi già esistenti tramite relazioni. Si sfrutta una query in CYPHER per potervi interagire, e sono presenti nello specifico i parametri:

- **Post**: Il testo generato dal LLM e revisionato, in aggiunta vi è la data di pubblicazione.
- **Topic**: L'argomento trattato dal Post, serve a collegare questi ultimi tra loro tramite la relazione `[:HAS_TOPIC]`. Tramite la relazione `[:RELATED_TO]`, essi sono tra loro associati tramite cross-similarity tra i loro embedding per un'eventuale "espansione semantica" in fase di ricerca.
- **Source**: Memorizza i tool utilizzati per la stesura, tracciandoli tramite `[:USES_SOURCE]`. Gli strumenti possono fare riferimento ai documenti già presenti nel KG o la ricerca Web, in base a quali sono stati utilizzati (vedere paragrafo Tool).
- **Claim**: Brevi frasi che danno una sintesi e concetti chiavi sull'argomento; sono stati realizzati sempre mediante l'uso dell'LLM e archiviati collegandoli tramite la relazione

[CONTAINS_CLAIM]. Anchessi presentano embeddig vettoriali per confronto tra post simili.



Un'altra funzione `save_editorial_plan()` viene utilizzata, adoperando sempre una query in CYPHER, per memorizzare il pieno editoriale una volta approvato dall'utente; i parametri utilizzati nel nodo sono i seguenti:

- **Week Key:** per poter indicizzare si utilizza una chiave, composta dal numero della settimana e dall'anno; in questo modo si va a verificare se è già presente una schedulazione programmata per quei sette giorni.
- **Plan Text:** Rappresenta il contenuto del piano approvato. La struttura prevede il giorno previsto di pubblicazione, il topic giornaliero e una breve descrizione dello stesso.
- **Curr_Date:** La data in cui è stato creato il plan.

Questo Knowledge Graph viene interrogato in modo attivo dal sistema, in particolare in tre momenti del ciclo:

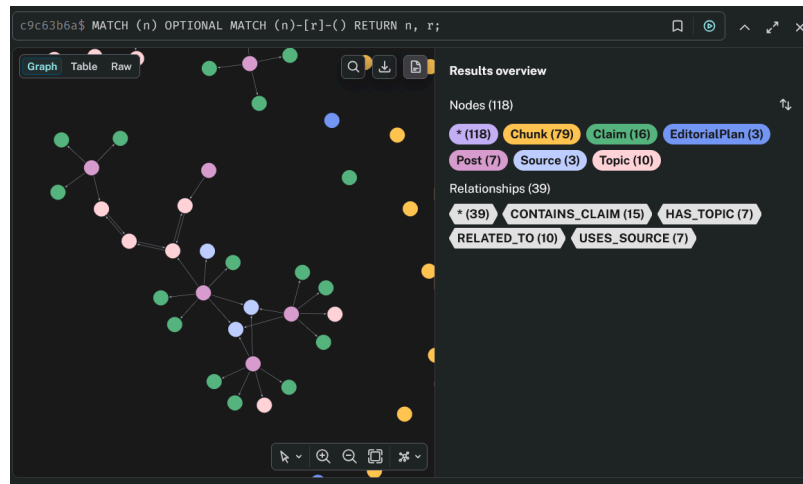
- In fase di Pianificazione Settimanale: il nodo `planner_node` utilizza la funzione `query_graph_plan()`. Questa si occupa di ricercare nel KG se è già presente un Plan relativo alla settimana corrente sulla base della precedente discussa Week Key. Ciò serve a evitare di riprogrammare l'ordine dei post ogni qual volta si faccia partire il programma. Se risulta presente, si verifica se oggi è "giorno di pubblicazione", confrontando la data di oggi con quelle elencate nel piano (`is_publish_day()`). Qualora la giornata di oggi non fosse presente in elenco, il programma lo stamperà a video e concluderà per tempo il proprio lavoro; alternativamente si andrà in fase di Pianificazione POST, estraendo il topic relativo.
- In fase di Pianificazione Settimanale: se non viene trovato il piano, allora si passa alla stesura dello stesso alla successiva revisione, per poi alla fine salvarlo tramite `save_editorial_plan()`.

Il nodo `planner_node` utilizza la funzione `suggest_next_topics()`. Questa si occupa di suggerire il topic di cui tratterà il post giornaliero. Per far ciò si sfruttano altre due funzioni:

- `get_recent_topics()` esegue una query in cui si estraggono gli ultimi 30 argomenti trattati, in modo da evitare ripetizioni.
- `get_topic_gaps()` esegue una query per individuare i topic già inseriti nel grafo, ma non ancora associati a nessun post. Il risultato è un cosiddetto "gap di copertura".

In base ai risultati delle due funzioni, verranno passati come topic i 10 meno recenti e/o non ancora coperti. In alternativa saranno inviati dei “topic generici” già preimpostati di “fallback” (Legione, Gladiatori, Monete...)

- Nella fase di Redazione del Post: dei tool specializzati si occupano di interrogare il KG tramite funzione *query_knowledge_graph()* (collegata alla successiva *query_graph_rag()*) per recuperare il contesto storico e i documenti. Questi verranno forniti all'LLM come supporto per la generazione del contenuto, cercando di garantire anche una continuità tematica coerente.



II K-RAG

Il **Knowledge Graph-augmented RAG** è una strategia di “recupero della conoscenza”, con la quale si combina la ricerca semantica sui documenti con le informazioni già presenti nel Knowledge Graph. Per poter realizzare il post infatti, viene utilizzata la funzione *query_graph_rag()*, in modo da accedere al grafo in cerca di documenti utili alla stesura.

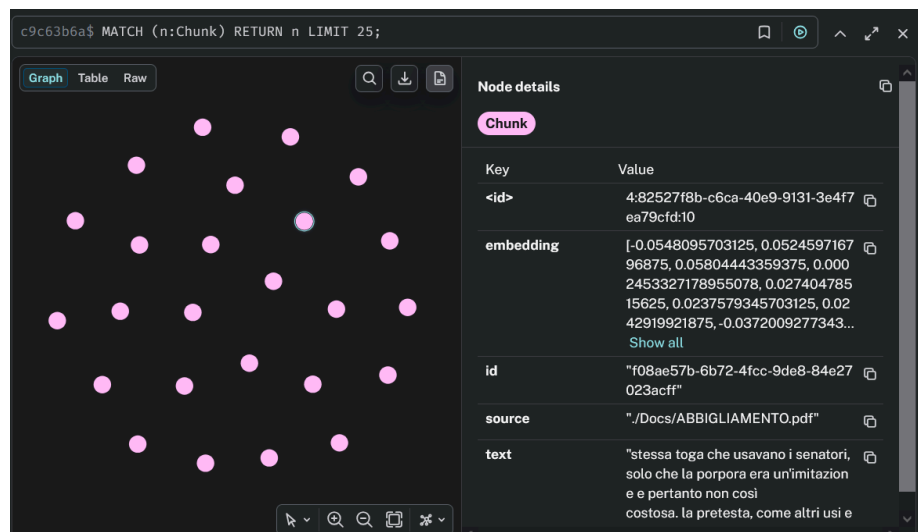
Prima di avviare la ricerca, mediante *get_related_topics()* si cercano eventuali topic correlati a quello scelto per il testo. Se va a buon fine, la query originale viene espansa aggiungendovi i concetti collegati, per ottenere una rappresentazione più completa e ampia del tema.

Questa query, espansa o non, viene sfruttata dal **VectorRetriever**, un componente di Neo4j il quale effettua una ricerca semantica sui chunk dei documenti pre-caricati e memorizzati sotto forma di embedding vettoriali basandosi sulla cosine-similarity per il confronto. Con ciò, il sistema riesce a recuperare i passaggi più pertinenti all'argomento e restituiti successivamente all'agente sottoforma di “contesto documentale”, ossia il chunk del documento in cui si parla dell'argomento previsto e la sua sorgente originaria.

Durante la generazione del post, l'LLM sfrutta questo contesto durante la stesura; in tal modo si cerca di ridurre il rischio di allucinazioni, il testo generato infatti dovrà basarsi su questa “ancora di verità” per evitare di scrivere frasi non supportate. Per garantire la massima trasparenza, nel prompt fornito all'LLM viene evidenziato di inserire esplicitamente i riferimenti nel testo, sia che esso sia il [KG-RAG], che un eventuale ricerca su Internet [Web-Search].

Il KG dunque non solo memorizza il piano editoriale e i post, ma anche dei documenti che fungono da fonte di verità, suddivisi in chunk. A livello pratico, il software ha, in fase di creazione, caricato 4 documenti PDF riguardanti alcuni argomenti della Roma Antica da usare per i post (Breve biografie di imperatori, elenco delle legioni, breve tesi sul vestiario e sulla numismatica).

I testi sono stati suddivisi in chunk da 1500 caratteri (con un overlap di 150 per non perdere il contesto tra i “frammenti”) e sequenzialmente processati in modo asincrono. Nel codice si è posta una pausa tra un processo e l'altro per rispettare i Rate Limits imposti dalle API di embedding di MistralAI (*mistral-embed*).



Tooling

Un sistema agentico, per sua natura, è in grado di utilizzare diversi **tool** messi a sua disposizione per ottimizzare i task e sfruttare strumenti in modo dinamico.

Nel sistema in questione, sfruttando **LangGraph** (framework orchestrativo usato per la creazione di applicazioni multi-agent), sono stati implementati un set di tool specializzati:

- *search_web*: è un tool di ricerca Web usato per prelevare informazioni da Internet e integrarle nei dati per la stesura del post. Si è utilizzato come motore di ricerca **DuckDuckGo** per ragioni di semplicità di utilizzo e costo. Tramite una lista di siti “trusted”, si è imposto inoltre un filtro in modo da limitare i risultati ed evitare di utilizzare fonti qualsiasi (es. *Wikipedia*, *Treccani*, ecc...).
- *query_knowledge_graph*: oltre a cercare sul Web, mediante questo tool si cercano le informazioni relative al topic del giorno anche tra i documenti conservati nel KG.
- *fact_checker*: il tool si occupa di unire le informazioni prelevate dai due precedenti per validarne le informazioni prima di essere usate nel post.
- *suggest_topic*: durante la pianificazione settimanale, mediante questo tool si va a risalire a quelli precedentemente trattati; questa cronologia sarà usata per suggerire topic che non sono stati ancora trattati o per quelli con post più vecchi, per evitare ripetizioni e ridondanze.

- *week_today*: tool molto semplice che fornisce il giorno della settimana corrente, serve a verificare, se per oggi sono programmati post oppure no e per dare contesto anche all'LLM relativo alla parte museale.
- *soldium_imperium*: questo tool si interfaccia con un modello di Visual Transformer fine-tuned sulla monetazione romana. Verrà usato qualora il topic della giornata riguardi la moneta: andrà a prelevare mediante numero pseudo-casuale un conio dal database e lo passerà al modello; il risultato della chiamata fornirà all'LLM il nome dell'imperatore in essa raffigurato sulla quale figura e riforme monetarie sarà incentrato il post seguente.

A livello di codice i vari tool sono stati orchestrati e agganciati mediante *llm.bind_tools()*. Si è imposto poi il parametro *tool_choice* ad "auto" (tranne nel caso usasse il ViT) in modo che l'LLM decida in autonomia quando e quale strumento utilizzare (anche se via prompt si è imposto di dare preferenza all'utilizzo del KG per raccogliere le informazioni).

I messaggi di risposta del modello che richiedono l'uso di un tool vengono poi eseguiti fisicamente dal nodo *ToolNode(tools)* di LangGraph. Per avere ulteriore trasparenza, tramite i messaggi di tool vengono registrati nel nodo di *save_tool_output()*.

Reasoning e ReACT

Nel sistema inoltre è stato implementato il paradigma **ReACT**: il modello alterna fasi di ragionamento e azione, adottando tecniche di prompting "step by step" per la risoluzione.

Il prompt, come previsto dal paradigma, impone all'LLM di strutturare la risposta secondo i passaggi **Thought** (viene analizzata la situazione e si pianifica il passo successivo in base alla motivazione); **Action** (si esegue l'operazione mediante l'uso dei tool) ed **Observation** (vengono raccolti i risultati delle azioni compiute).

Da questo viene creata la lista **reasoning_trace**, la quale tiene traccia di tutte le giustificazioni e l'esito delle varie chiamate ai tool. La lista è inoltre salvata nello stato **AgentState** per essere sempre disponibile e consultabile; si tratta di una struttura basata su *TypedDict* che funge da "memoria centralizzata" del grafo. Ogni volta che un nodo viene eseguito, esso restituisce un aggiornamento che modifica lo stato globale, rendendo la *reasoning_trace* accessibile e modificabile in tempo reale da qualsiasi punto del flusso.

Dopo l'approvazione finale del post, viene offerta all'utente la possibilità di stampare a schermo l'intera cronologia di questa traccia per avere un feedback visivo sul comportamento dell'agente.

State Management

Il sistema sfrutta la classe *AgentState*, oltre per la *reasoning_trace*, anche per memorizzare l'input dell'utente (*messages*), i risultati dei vari tool invocati (*tool_outputs*), le info di pianificazione (*planning_info*) e i vari contesti per il KG (*kg_summary*, *claims*, *sources*).

Grazie all'architettura LangGraph, questo stato viene aggiornato dinamicamente al termine di ogni singolo nodo. Questo flusso continuo propaga il contesto aggiornato ai nodi successivi, ottimizzando le decisioni dei router e l'output finale dell'LLM.

Human in the Loop Mechanism

Ogni post e programmazione che l'LLM ha generato, prima di essere pubblicato deve passare per una revisione umana, in un meccanismo "**Human-in-the-loop**". Questo è il principio per cui un sistema, durante il suo workflow, è sottoposto ad un processo decisionale o di supervisione da parte di un essere umano, specie in contesti di precisione, sicurezza o etica.

Nel sistema in questione ciò accade, come scritto precedentemente, in due fasi.

Una volta generato l'EditorialPlan, il grafo utilizza la funzione *interrupt()* per fermarne temporaneamente l'esecuzione e viene chiesto all'utente (nel ruolo dell'amministratore del blog) cosa ne pensa del risultato e il perché. Viene poi ripreso il flusso tramite l'invio di un oggetto **Command** e attivato il nodo *hitl_plan_node()* il quale riceve e analizza la risposta dell'utente.

- Se risposto "approve", il piano viene salvato nel KG mediante apposita chiamata a *save_editorial_plan()*, passando poi o alla creazione del post qualora fosse un giorno di pubblicazione (flusso all'*assistant()*), o direttamente al nodo di terminazione (evitando di generare post quando non necessario).
- Se invece fosse "reject", viene riportato al nodo *planner()* insieme al feedback espresso; LLM verrà indirizzato a riscrivere il piano tenendo conto delle specifiche e dell'impostazione prestabilita, ma applicando le modifiche in base a ciò che l'admin ha richiesto. Ciò è possibile dato che si sta iniettando il messaggio d'errore (**HumanMessage**) nel prompt.

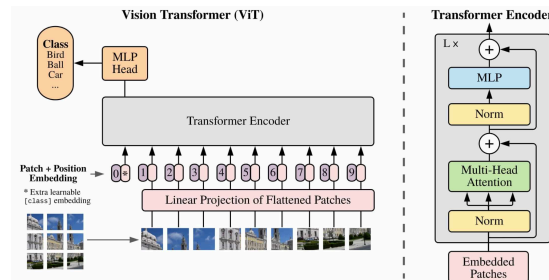
Per i post il meccanismo è molto simile, solo che viene gestito dal nodo *hitl_node()*. Una volta generato il post si passa prima per un controllo dei claim (vengono analizzati i claim prodotti con quelli già presenti nel KG tramite **cross-similarity**, se risultati troppo simili il post verrà riscritto), e poi alla revisione. In modo analogo al caso precedente si valutano tre possibili casi:

- Se risposto "approve", il post (e relativi claim e source(nodi)) viene salvato nel KG mediante apposita chiamata ad *update_kg_node()*.
- Se risposto "reject", il controllo ritorna all'assistente, passandogli contesto il feedback testuale inserito dall'utente per la riscrittura, tenendo comunque conto delle specifiche precedenti.
- Se risposto "modify" si permette all'utente di riscrivere completamente a mano il post, sovrascrivendo il precedente all'interno dell'AgentState.

Fine-Tuned Component

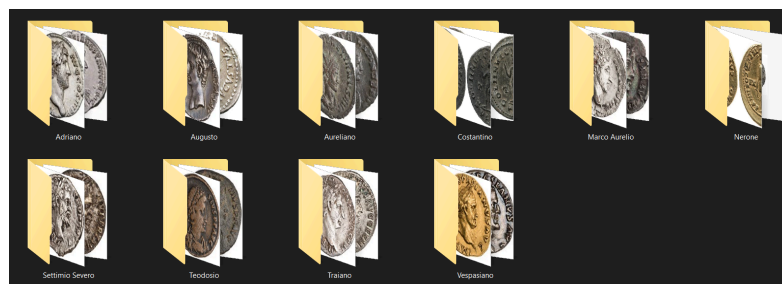
Insieme all'agente principale è stato sviluppato anche un classificatore di immagini basato su **Vision Transformer**. Il modello ViT viene richiamato dall'agent in inferenza tramite il tool *soldium_imperium*. Quando il topic giornaliero è "Monete", esso viene richiamato a riconoscere, a partire dall'immagine di una moneta romana, l'imperatore raffigurato e restituirne il nome; successivamente verrà generato un post dedicato a quell'imperatore, fornendone una breve biografia e le informazioni sul sistema monetario durante il suo regno.

Dal punto di vista tecnico è stato utilizzato come base il modello **google/vit-base-patch16-224**, un ViT pre-addestrato su dataset ImageNet il quale accetta immagini 224×224 pixel da poi suddividere in patch 16×16 , processate trasformandole in token e mandate a elaborare (con un token CLS) dai layer del Transformer tramite meccanismo di self-attention. Per poter adattare al meglio questo modello alla classificazione è stata utilizzata la classe **ViTForImageClassification**, la quale sostituisce la classification head finale con una nuova.



Il dataset utilizzato è un dataset custom composto da circa 200 immagini di monete romane, distribuite in 10 classi corrispondenti a differenti imperatori in modo bilanciato. Prima di essere fornite al modello, le immagini subiscono un pre-processing tramite **ViTImageProcessor**; le fotografie vengono ridimensionate, normalizzati i canali e convertite.

Oltre al processing, il dataset è stato suddiviso nelle tre sezioni fondamentali: il **Training Set (70%)** per l'aggiornamento dei pesi; il **Validation Set (15%)**, per selezionare degli iperparametri e infine il **Test Set (15%)** per la valutazione finale.



In tema di valutazione, per poter misurare le prestazioni del classificatore sono state monitorate diverse metriche: **Accuracy, Precision, Recall** ed **F1 Score**.

Addestramento: Full Fine-Tuning

Date queste premesse, prima di poter essere usato in modo diretto, il ViT è stato sottoposto ad un processo di fine-tuning e ad un'analisi e confronto con il modello base.

In una prima fase è stato utilizzato il modello senza alcun fine-tuning come baseline e valutato. Come facilmente prevedibile, il risultato evidenzia prestazioni molto limitate, con accuracy molto bassa e distribuzioni di probabilità abbastanza uniformi tra le varie classi.

Si è poi effettuato un processo di **full fine-tuning**. In questo modo tutti i pesi sono stati aggiornati e il modello è stato completamente addestrato.

Per individuare quale fosse la configurazione migliore per quel task, sono stati eseguiti diversi esperimenti di **hyperparameter search** sul modello. Nel dettaglio sono state eseguite tre varianti modificando il learning rate, il numero di epoch ed il batch size. Ognuna di queste configurazioni è stata addestrata in modo indipendente dagli altri e poste a confronto:

- N1: Learning Rate = 3^{-4} , Epochs = 1, Batch Size = 8
- N2: Learning Rate = 1^{-4} , Epochs = 2, Batch Size = 4
- N3: Learning Rate = 3^{-5} , Epochs = 4, Batch Size = 4

Dai risultati (vedere capitolo finale) è emerso che se usato un learning rate troppo elevato (3^{-4}), le prestazioni risultano inferiori. Anche l'aumento di epoche non ha portato miglioramenti particolarmente significativi. Considerando l'accuracy e l'F1-score ottenute sui vari esperimenti e il costo computazionale corrispondente, si è scelta la configurazione N2.

Essa ha raggiunto un accuracy pari a circa il 30% (40% in alcuni casi in cui è stata rifatta l'analisi); non troppo alta nel complesso, ma comunque migliore rispetto alle sue concorrenti. Anche osservando la loss durante il training è stato possibile notarne una diminuzione, confermando che il modello ha effettivamente appreso informazioni rilevanti dal dataset.

Addestramento: Tentativo Parameter-Efficient Fine-Tuning

Per ottimizzare ancora meglio il modello, si è inoltre stato testato un approccio **PEFT** utilizzando la tecnica **LoRA** su N2. In questo modo invece di aggiornare le matrici weight originali, si congelano e si sfrutta la variazione del peso come prodotto di due matrici di basso rango; riducendo il numero di parametri addestrabili. LoRA è stato iniettato nei moduli di linear attention e con un rango 16.

All'atto pratico LoRA ha registrato prestazioni inferiori rispetto al FFT, con un'accuratezza e F1-score a circa 0.06. Questo può essere riconducibile al fatto che il dataset è ridotto e il congelamento della *backbone* impedisce un adattamento completo e accettabile. Di conseguenza, la scelta finale è ricaduta sul modello N2-Full Fine-Tuning.

Test Finale e Attention Rollout

In ultima fase, il modello fine-tunato migliore è stato valutato andando a verificare le metriche sul Validation Set e sul Test Set; nonché verificato anche con sulla stessa img di prova usata con la baseline.

Oltre alla valutazione mediante metriche, è stato implementato anche un meccanismo di **Attention Rollout** per avere anche una accortezza visiva sui criteri decisionali. Sul modello scelto viene eseguita la funzione *prediction()*, il quale fa una semplice inferenza e restituisce le classi più probabili insieme alle relative probabilità. Parallelamente, la funzione *attention()* calcolata la media delle matrici di attenzione per ogni layer, per poi eseguire normalizzazione, rollout (ricorsione) ed estrazione del token [CLS].

La heatmap risultante viene ridimensionata e sovrapposta all'immagine originale, evidenziando le zone della moneta che hanno maggiormente contribuito alla decisione del classificatore.



Come ultimo step, il modello viene salvato tramite `save_pretrained()`, insieme al relativo `ViTImageProcessor` e alla versione finale del dataset, per essere utilizzato dalla funzione di inferenza `inferenza()` richiamata dal tool. Questa, come brevemente accennato all'inizio, prende un'immagine dal Test set, applica lo stesso preprocessing per adattarla e la passa al modello FT; il risultato (nome dell'imperatore con la probabilità più alta) sarà restituita al main-agent.

Final Evaluation

Di seguito verrà proposto il funzionamento dell'agente, con qualche considerazione personale sui risultati:

- 1) **AVVIO:** All'avvio del programma verifico se vi è un EditorialPlan per questa settimana. Ecco cosa succede se è presente ma non è giorno di pubblicazione.

In questo caso è stato provato di Giovedì, data di non pubblicazione. Ho anche fatto stampare a video il Trace per visualizzare tutto il ragionamento dell'agente.

```
BENVENUTO SU BLOG-MAKER (ROMA EDITON) MUSEO!

Piano editoriale numero 26_2026 trovato

Oggi non è giorno di pubblicazione

nodo [planner]

Il lavoro quì è terminato, grazie e arrivederci.
Vuoi vedere il Trace finale?: si

REASONING TRACE FINALE

Thought: Recuperato piano editoriale corrente
Thought: Oggi non è un giorno di pubblicazione.
```

- 2) AVVIO: All'avvio del programma verifico se vi è un EditorialPlan per questa settimana. Ecco cosa succede se è presente ed è giorno di pubblicazione.

In questo caso verifica e trova sia il Plan, sia il topic di oggi: Religione Romana, esegue sia la ricerca nel KG (non trovando molto), e poi la WebSearch, come indicato sia dai log che dal Post stesso.

```
(C:\vivo) PS D:\utente\CartellaCodici\AI\AGENTS> python Amain.py
BENVENUTO SU BLOG-MAKER (ROMA EDITOR) MUSEO!

Piano editoriale numero 26_2026 trovato

Il Topic di oggi è Religione
Entrato nel nodo: [planner]
Entrato nel nodo: [assistant]
suggest_topics
Entrato nel nodo: [tools]
Entrato nel nodo: [save_tool_output]
Entrato nel nodo: [assistant]
query_knowledge_graph
Received notification from DBMS server: <GqlStatusObject gql_status='01N01', status_description='warn: feature deprecated with replacement. db.index.vector.queryNodes is deprecated. It is replaced by SEARCH.', position=<SummaryInputPosition line=1, column=1, offset=0>, raw_classification='DEPRECATION', classification=<NotificationClassification.DEPRECATION', severity='WARNING', 'pos
ition': {'offset': 0, 'line': 1, 'column': 1}, 'OPERATION': '', 'OPERATION_CODE': '0', 'CURRENT_SCHEMA': '/'> for query: 'CALL db.index.vector.queryNodes($vector_index_name, $to
p_k * $effective_search_ratio, $query_vector) YIELD node, score WITH node, score LIMIT $top_k RETURN node {.text, .source} AS node, labels(node) AS nodeLabels, elementId(node) AS
```

```
elementId, elementId(node) AS id, score'
Entrato nel nodo: [tools]
Entrato nel nodo: [save_tool_output]
Entrato nel nodo: [assistant]
Avviata anche la ricerca web
Entrato nel nodo: [tools]
Entrato nel nodo: [save_tool_output]
CLM:
['I sacerdoti romani, come i Flaminii e il Pontifex Maximus, erano figure centrali nella religione romana.', 'Le Vestali, sacerdotesse vergini, mantenevano il fuoco sacro di Roma e la sua prosperità.', 'Le Flaminiche partecipavano ai rituali indossando abiti specifici come la stola e il flammeum.']
Entrato nel nodo: [assistant]
Eseguire validazione
Entrato nel nodo: [claim_validator]
```

ECCO IL POST GENERATO

La religione romana era un sistema complesso in cui i sacerdoti giocavano un ruolo centrale. Tra questi, i Flaminii erano i più importanti: dedicati al culto di una singola divinità, come il Flamine di Giove, erano scelti tra le famiglie patrizie e indossavano un copricapo distintivo chiamato "apex". Un altro ruolo chiave era quello del Pontifex Maximus, la massima autorità religiosa, spesso ricoperto dall'imperatore stesso, che sovrintendeva ai rituali e al calendario sacro [Web-Search: Wikipedia].

Accanto a loro, le Vestali, sacerdotesse vergini dedite al culto di Vesta, erano responsabili di mantenere acceso il fuoco sacro della città. La loro purezza era essenziale per la prosperità di Roma. Anche le Flaminiche, mogli dei Flaminii, partecipavano ai rituali, indossando abiti specifici come la "stola" e il "flammeum" [Web-Search: Wikipedia].

Oggi il museo ti aspetta dalle 9 alle 21.

Il Post generato è, a mio giudizio, abbastanza generico, almeno per questo topic. In casi in cui le informazioni provenivano dal KG si è dimostrato più dettagliato; vedere successivamente al punto 5

Nel riprovare si è verificato cosa succedeva se era presente un claim considerato duplicato: la rigenerazione occupava molti cicli di riscrittura, facendo ripartire il programma continuava ma con meno cicli, l'ipotesi è che dipenda da come lo LLM rigenera il post e da quanto il tema sia più o meno ampio.

```
CLM:
['La religione romana rifletteva la società e la politica di Roma.', 'La Triade Capitolina simboleggiava unità politica e religiosa.', 'I Romani integravano divinità straniere tramite sincretismo religioso.']
nodo [assistant]
Eseguire validazione
Claim duplicato: 'La religione romana era un elemento centrale della vita pubblica e privata con un politeismo pratico.' (Sim: 0.96)
nodo [claim_validator]
CLM:
['La religione romana era un sistema dinamico che si evolveva con l'espansione di Roma.', 'I flaminii erano sacerdoti che gestivano culti ufficiali, tra cui quello della Triade Capitolina.', 'I Romani praticavano culti domestici per Penati e Lari, spiriti protettori della famiglia.']
nodo [assistant]
Eseguire validazione
Claim duplicato: 'La religione romana era un elemento centrale della vita pubblica e privata con un politeismo pratico.' (Sim: 0.96)
nodo [claim_validator]
```

- 3) AVVIO: All'avvio del programma verifico se vi è un EditorialPlan per questa settimana. Ecco cosa succede se non è presente.

```
BENVENUTO SU BLOG-MAKER (ROMA EDITON) MUSEO!

Nessun piano trovato nel KG. Generazione nuovo piano.

Piano generato, ma oggi non è giorno di pubblicazione.

nodo [planner]

ECCO IL PIANO EDITORIALE
Lunedì: Commercio
Breve descrizione: L'importanza delle rotte commerciali nell'Impero Romano e il loro impatto sull'economia.

Mercoledì: Persiani
Breve descrizione: I rapporti diplomatici e militari tra l'Impero Romano e l'Impero Persiano durante l'antichità.

Sabato: Religione
Breve descrizione: L'evoluzione e la diffusione delle religioni nell'Impero Romano, con focus su culti e sincretismi.
nodo [__interrupt__]

Il flusso si è fermato, ecco cosa era stato generato.
Cosa ne pensi? reject
Perchè?: i topic devono essere ca,biati, includendo: Colosseo, Anarchia del III Secolo, Vestiario
```

Il piano è stato generato correttamente, ma in questo caso decido di cambiare tutti i topic (lasciando stare l'errore di battitura).

```
Piano Editoriale rifiutato, avviare ri-pianificazione

nodo: [human_plan_review]
Nessun piano trovato nel KG. Generazione nuovo piano.

Piano generato, ma oggi non è giorno di pubblicazione.

nodo: [planner]

ECCO IL PIANO EDITORIALE
Lunedì: Colosseo
Breve descrizione: L'importanza storica e architettonica del Colosseo nell'Impero Romano, nonché il suo ruolo nella società e nell'intrattenimento pubblico.

Mercoledì: Anarchia del III Secolo
Breve descrizione: Le cause, gli eventi principali e le conseguenze dell'Anarchia del III Secolo nell'Impero Romano, un periodo di crisi politica e militare.

Sabato: Vestiario
Breve descrizione: L'evoluzione del vestiario nell'Impero Romano, con particolare attenzione ai simboli sociali, culturali e politici che esso rappresentava.
nodo: [__interrupt__]

Il flusso si è fermato, ecco cosa era stato generato.
Cosa ne pensi? approve
Perchè?: ok
```

Una volta approvato verrà salvato, ma dato che non deve essere pubblicato nulla ecco cosa succede.

```
Ok, il Piano Editoriale è approvato

nodo: [human_plan_review]

Il lavoro qui è terminato, grazie e arrivederci.
Vuoi vedere il Trace finale?: sì

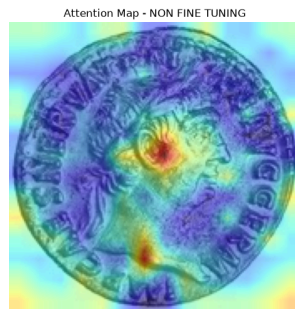
REASONING TRACE FINALE

Thought: Generato nuovo pianoeditoriale. Oggi non è un giorno di pubblicazione.
User: Piano Rifiutato. Note: i topic devono essere ca,biati, includendo: Colosseo, Anarchia del III Secolo, Vestiario
Thought: Generato nuovo pianoeditoriale. Oggi non è un giorno di pubblicazione.
User: Piano Approvato.
(.venv) PS D:\utente\CartellaCodici\AI\AGENT> []
```

Dato che oggi non è giorno di pubblicazione, non verrà generato niente, solo il Plan.

4) ViT: Ecco come sono stati i risultati dell'addestramento del Visual Transformer.

Prima di tutto si è verificato, modificando le label, su una baseline come veniva categorizzata una moneta di test, per poi visualizzare le metriche su tutto il test-dataset. I risultati non sono ovviamente elevati, essendo il modello non ancora addestrato.



```
Test 1 modello non fine-tunato su un' img a caso

PREDIZIONI
0.13750 : Traiano
0.11707 : Augusto
0.11699 : Settimio Severo
0.11493 : Costantino
0.11373 : Vespasiano
100%|

Log delle metriche pre training sul test-dataset
***** test metrics *****
epoch = 0
eval accuracy = 0.1333
eval f1-score = 0.1333
eval loss = 2.2948
eval model_preparation_time = 0.0031
eval precision = 0.1333
eval recall = 0.1333
eval runtime = 0:00:00.04
eval samples_per_second = 46.611
eval_steps_per_second = 6.215
```

Si è poi andato a fare una serie di Full-Fine Tuning per scoprire i parametri migliori: sono stati fatti tre esperimenti modificando Learning Rate, Epoch e Batch Size:

N1: $l_rate = 3^{-4}$, epoch = 1, $b_size = 8$

```
PROVA N1
Resolving data files: 100%| 200/200 [00:00:00:00, 14142.951it/s]
[transformers] You passed 'num_labels=10' which is incompatible to the 'id2label' map of length '1000'.
Loading weights: 100%| 200/200 [00:00:00:00, 6220.881it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key | Status |
-----|-----|
classifier.bias | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size([10])
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.Size([10, 768])
Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
[transformers] You passed 'num_labels=10' which is incompatible to the 'id2label' map of length '1000'.
Loading weights: 100%| 200/200 [00:00:00:00, 5563.951it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key | Status |
-----|-----|
classifier.bias | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size([10])
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.Size([10, 768])
Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
{'eval_loss': 2.257, 'eval_accuracy': 0.1333, 'eval_precision': 0.1333, 'eval_recall': 0.1333, 'eval_f1-score': 0.1333, 'eval_runtime': 0.200, 'eval_samples_per_second': 159, 'eval_steps_per_second': 20, 'epoch': 1}
Writing model shards: 100%| 1/1 [00:00:00:00, 1.891it/s]
{'train_runtime': 3.508, 'train_samples_per_second': 39.91, 'train_steps_per_second': 5.131, 'train_loss': 2.396, 'epoch': 1}
100%| 4/4 [00:00:00:00, 5.131it/s]
100%| 4/4 [00:00:00:00, 31.991it/s]
```

ESPERIMENTO N1

```
{'eval_loss': 2.2574055194854736, 'eval_accuracy': 0.13333333333333333, 'eval_precision': 0.13333333333333333, 'eval_recall': 0.13333333333333333, 'eval_f1-score': 0.13333333333333333, 'eval_runtime': 0.1882, 'eval_samples_per_second': 159.402, 'eval_steps_per_second': 21.254, 'epoch': 1.0}
```

N2: $l_rate = 1^{-4}$, epoch = 2, $b_size = 4$

```
PROVA N2
Resolving data files: 100%| 200/200 [00:00:00:00, 11418.661it/s]
[transformers] You passed 'num_labels=10' which is incompatible to the 'id2label' map of length '1000'.
Loading weights: 100%| 200/200 [00:00:00:00, 6251.431it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key | Status |
-----|-----|
classifier.bias | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size([10])
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.Size([10, 768])
Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
[transformers] You passed 'num_labels=10' which is incompatible to the 'id2label' map of length '1000'.
Loading weights: 100%| 200/200 [00:00:00:00, 5932.161it/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key | Status |
-----|-----|
classifier.bias | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size([10])
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.Size([10, 768])
Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
{'eval_loss': 1.924, 'eval_accuracy': 0.3, 'eval_precision': 0.3, 'eval_recall': 0.3, 'eval_f1-score': 0.3, 'eval_runtime': 0.2525, 'eval_samples_per_second': 118.8, 'eval_steps_per_second': 15.84, 'epoch': 2}
Writing model shards: 100%| 1/1 [00:00:00:00, 1.791it/s]
{'train_runtime': 7.58, 'train_samples_per_second': 36.94, 'train_steps_per_second': 9.235, 'train_loss': 1.923, 'epoch': 2}
100%| 70/70 [00:07:00:00, 9.241it/s]
100%| 4/4 [00:00:00:00, 31.791it/s]

ESPERIMENTO N2
{'eval_loss': 1.923577904701233, 'eval_accuracy': 0.3, 'eval_precision': 0.3, 'eval_recall': 0.3, 'eval_f1-score': 0.3, 'eval_runtime': 0.1953, 'eval_samples_per_second': 153.617, 'eval_steps_per_second': 20.482, 'epoch': 2.0}
```

N3: $l_rate = 3^{-5}$, epoch = 4, $b_size = 4$

```

PROVA N3
Resolving data files: 100% | 200/200 [00:00<00:00, 13088.591t/s]
[transformers] You passed 'num_labels=10' which is incompatible to the 'id2label' map of length '1000'.
Loading weights: 100% | 200/200 [00:00<00:00, 5478.781t/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key | Status |
-----
classifier.bias | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size([10])
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.Size([10, 768])

Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
[transformers] You passed 'num_labels=10' which is incompatible to the 'id2label' map of length '1000'.
Loading weights: 100% | 200/200 [00:00<00:00, 6416.251t/s]
[transformers] ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224
Key | Status |
-----
classifier.bias | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000]) vs model:torch.Size([10])
classifier.weight | MISMATCH | Reinit due to size mismatch - ckpt: torch.Size([1000, 768]) vs model:torch.Size([10, 768])

Notes:
- MISMATCH: ckpt weights were loaded, but they did not match the original empty weight shapes.
{'loss': '1.693', 'grad_norm': '9.638', 'learning_rate': '0.786e-06', 'epoch': '2.857'}
{'eval_loss': '1.947', 'eval_accuracy': '0.2333', 'eval_precision': '0.2333', 'eval_recall': '0.2333', 'eval_f1-score': '0.2333', 'eval_runtime': '0.1852', 'eval_samples_per_second': '162', 'eval_steps_per_second': '21.6', 'epoch': '2.857'}
{'eval_loss': '1.888', 'eval_accuracy': '0.2667', 'eval_precision': '0.2667', 'eval_recall': '0.2667', 'eval_f1-score': '0.2667', 'eval_runtime': '0.1764', 'eval_samples_per_second': '170.1', 'eval_steps_per_second': '22.67', 'epoch': '4'}
Writing model shards: 100% | 1/1 [00:00<00:00, 1.981t/s]
[train runtime': '13.56', 'train samples per second': '41.3', 'train steps per second': '10.32', 'train loss': '1.421', 'epoch': '4']
100% | 140/140 [00:13<00:00, 10.321t/s]
100% | 4/4 [00:00<00:00, 28.951t/s]

ESPERIMENTO N3
{'eval_loss': '1.8879228830337524', 'eval_accuracy': '0.26666666666666666', 'eval_precision': '0.26666666666666666', 'eval_recall': '0.26666666666666666', 'eval_f1-score': '0.26666666666666666', 'eval_runtime': '0.2152', 'eval_samples_per_second': '139.412', 'eval_steps_per_second': '18.588', 'epoch': '4.0'}

```

RISULTATI

```

N1 --> {'eval_loss': '2.2574055194854736', 'eval_accuracy': '0.13333333333333333', 'eval_precision': '0.13333333333333333', 'eval_recall': '0.13333333333333333', 'eval_f1-score': '0.13333333333333333', 'eval_runtime': '0.1882', 'eval_samples_per_second': '159.402', 'eval_steps_per_second': '21.254', 'epoch': '1.0'}
N2 --> {'eval_loss': '1.923577904701233', 'eval_accuracy': '0.3', 'eval_precision': '0.3', 'eval_recall': '0.3', 'eval_f1-score': '0.3', 'eval_runtime': '0.1953', 'eval_samples_per_second': '153.617', 'eval_steps_per_second': '20.482', 'epoch': '2.0'}
N3 --> {'eval_loss': '1.8879228830337524', 'eval_accuracy': '0.26666666666666666', 'eval_precision': '0.26666666666666666', 'eval_recall': '0.26666666666666666', 'eval_f1-score': '0.26666666666666666', 'eval_runtime': '0.2152', 'eval_samples_per_second': '139.412', 'eval_steps_per_second': '18.588', 'epoch': '4.0'}

Il miglior modello 'Full-Fine-Tuned' = N2

```

Per ulteriore verifica si è preso il modello migliore FFT, (N2) e si è applicato LoRA

Lora Param: rango = 16, lora_alpha = 16

```

PROVA CON LORA
{'eval_loss': '2.423', 'eval_accuracy': '0.06667', 'eval_precision': '0.06667', 'eval_recall': '0.06667', 'eval_f1-score': '0.06667', 'eval_runtime': '0.2903', 'eval_samples_per_second': '103.3', 'eval_steps_per_second': '13.78', 'epoch': '2'}
{'train runtime': '6.249', 'train samples per second': '44.81', 'train steps per second': '11.2', 'train loss': '2.368', 'epoch': '2'}
100% | 70/70 [00:06<00:00, 11.201t/s]
100% | 4/4 [00:00<00:00, 16.841t/s]

ESPERIMENTO N2-LoRA
{'eval_loss': '2.4229490756988525', 'eval_accuracy': '0.06666666666666667', 'eval_precision': '0.06666666666666667', 'eval_recall': '0.06666666666666667', 'eval_f1-score': '0.06666666666666667', 'eval_runtime': '0.3712', 'eval_samples_per_second': '80.819', 'eval_steps_per_second': '10.776', 'epoch': '2.0'}

RISULTATI
N2 --> 0.3
N2 -LoRA --> 0.06666666666666667

```

Si ha un peggioramento dei risultati; possibilmente ciò è dovuto ai parametri scelti.

Determinato qual'è il modello migliore sulla base dei risultati, esso viene memorizzato e fatta una verifica delle metriche sul dataset Validation e Test.

```

VALUTAZIONE SUL MODELLO MIGLIORE
100% | 4/4 [00:00<00:00, 23.561t/s]

RISULTATI VALIDATION MIGLIORE:
***** final_validation metrics *****
epoch = 2.0
eval_accuracy = 0.3667
eval_f1-score = 0.3667
eval_loss = 1.8677
eval_precision = 0.3667
eval_recall = 0.3667
eval_runtime = 0:00:00.26
eval_samples_per_second = 114.594
eval_steps_per_second = 15.279
100% | 4/4 [00:00<00:00, 28.391t/s]

RISULTATI TEST MIGLIORE:
***** final_test metrics *****
epoch = 2.0
eval_accuracy = 0.4333
eval_f1-score = 0.4333
eval_loss = 1.7485
eval_precision = 0.4333
eval_recall = 0.4333
eval_runtime = 0:00:00.20
eval_samples_per_second = 143.141
eval_steps_per_second = 19.085

```



```
Validation accuracy: 0.3666666666666664
Validation precision: 0.3666666666666664
Validation recall: 0.3666666666666664

Test accuracy: 0.4333333333333335
Test precision: 0.4333333333333335
Test recall: 0.4333333333333335
```

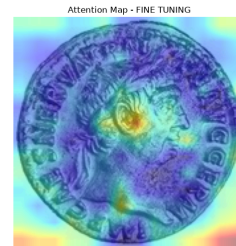
Infine si è passata la stessa moneta usata per testare la Baseline. Da far notare non solo il miglioramento anche se non ingente, della predizione, ma anche come è cambiata l'attention map. Ciò può essere spiegato per colpa di un dataset piccolo e con immagini molto simili tra una classe e un'altra. Inoltre a influire sulla predizione possono essere causa i parametri utilizzati, usandone altri potrebbero esserci miglioramenti (motivi di tempo e prestazioni mi hanno impedito ulteriori test).

Nell'img l'attenzione posta sull'angolo in basso a destra può essere causato da alcune img che presentano un watermark in quell'angolo, ma essendo distribuite abbastanza omogeneamente tra le classi, non gli imputo troppa rilevanza nella predizione (corretta).

Test 2 modello fine-tunato sulla stessa img a caso

PREDIZIONI

```
0.36847 : Traiano
0.30191 : Vespasiano
0.08479 : Adriano
0.08323 : Marco Aurelio
0.05269 : Nerone
```



5) ViT + Post: Utilizziamo i risultati del Visual Transformer per creare il Post.

Assumendo che il tema di oggi fossero le monete, esso il flusso fatto dall'agente il quale sfrutta come tool il ViT trattato in precedenza.

```
BENVENUTO SU BLOG-MAKER (ROMA EDITION) MUSEO!

Piano editoriale numero 26_2026 trovato

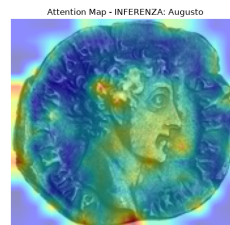
Il Topic di oggi è Monete
nodo [planner]
if monete print

nodo [assistant]
Chiamata al ViT

<PIL.JpegImagePlugin.JpegImageFile image mode=RGB size=175x161 at 0x1EC345F7850>

PREDIZIONI SU IMMAGINE 28:
1
0.26852 : Augusto
9
0.19930 : Vespasiano
0
0.08919 : Adriano
8
0.08353 : Traiano
5
0.07144 : Nerone

RISULTATO PIÙ ALTO: 0.26852, Augusto
```



```

nodo [tools]
nodo [save_tool_output]
if monete print

nodo [assistant]
nodo [tools]
nodo [save_tool_output]
if monete print

CLM:
["Augusto fu il primo imperatore romano, governando dal 27 a.C. al 14 d.C.", "Il Sesterzio, introdotto nel 23 a.C., fu una riforma monetaria simbolo di prosperità sotto Augusto", "Un soldato semplice guadagnava circa 900 sesterzi all'anno durante il regno di Augusto"]

nodo [assistant]
Eseguire validazione
nodo [claim_validator]

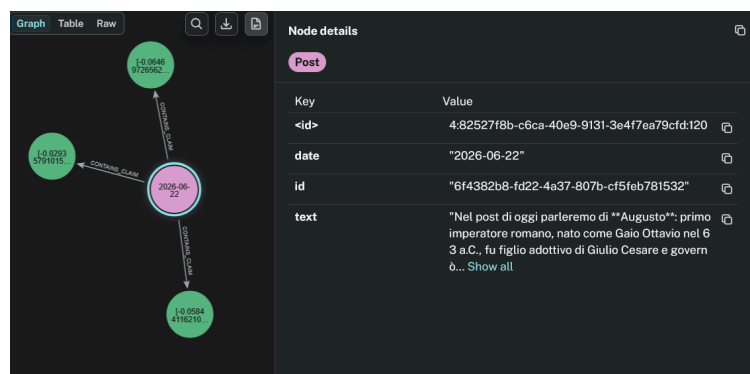
ECCO IL POST GENERATO
Nel post di oggi parleremo di Augusto: primo imperatore romano, nato come Gaio Ottavio nel 63 a.C., fu figlio adottivo di Giulio Cesare e governò Roma dal 27 a.C. al 14 d.C. Sotto il suo regno, l'impero conobbe un periodo di pace e prosperità noto come "Pax Augusta".

Nel nostro museo è presente una moneta coniata durante il suo regno: il Sesterzio, una delle principali riforme monetarie di Augusto introdotta nel 23 a.C.. Questa moneta, realizzata in "oricalco" (lega di rame e zinco), recava sul dritto il ritratto dell'imperatore e sul rovescio simboli del potere e della prosperità, come la scritta "SC" (Senatus Consultum), a indicare il legame con il Senato. Il sesterzio divenne una delle monete più diffuse, simbolo dell'economia stabile dell'epoca, dove un soldato semplice guadagnava circa 900 sesterzi all'anno [KG-RAG].

Oggi è lunedì, il museo ti aspetta dalle 9 alle 21.
nodo [_interrupt_]

Il flusso si è fermato, ecco cosa era stato generato.
Cosa ne pensi?

```



PS: Ricordarsi che le API KEY e le istanze gratuite di Neo4J dopo un pò scadono avrebbero risparmiato più di qualche arrabbiatura